

NiceGUI app.storage.browser 全面详细解析

`app.storage.browser` 是 NiceGUI 内置的五种核心存储类型之一，专为浏览器端会话级存储设计，核心特点是“数据存储在浏览器 Cookie 中、用户级跨标签页共享、无服务器持久化依赖”，适用于存储轻量、非敏感的用户会话数据（如临时标识、简单偏好设置），是平衡“客户端本地存储”与“跨标签页共享”的轻量方案。

一、核心定位与设计目标

1. 核心价值

- 浏览器端本地存储：**数据直接存储在用户浏览器的会话 Cookie 中，无需服务器磁盘 / 内存占用，减轻服务器压力；
- 用户级跨标签页共享：**同一用户的同一浏览器下，所有标签页可共享数据（Cookie 全局可见），适配跨标签页协作场景；
- 无服务器依赖：**数据读写仅在客户端完成，不与服务器交互（除首次生成用户 ID 外），响应速度快；
- 轻量易用：**API 模仿 Python 字典，支持键值对增删改查，无需手动处理 Cookie 序列化 / 解析；
- 身份标识基础：**默认存储浏览器会话唯一 ID（`app.storage.browser['id']`），为 `app.storage.user` 提供用户身份标识基础。

2. 设计目标

- 解决“浏览器端轻量数据存储”需求，避免将简单临时数据存入服务器存储（如 `app.storage.user`）；
- 支持跨标签页共享数据（如用户临时选择的状态、会话标识），提升客户端交互体验；
- 简化 Cookie 操作：框架自动处理数据序列化、Cookie 写入 / 读取，无需手动编写 Cookie 相关代码；
- 适配“短期会话、非敏感数据”存储场景，补充服务器端存储的不足。

二、核心特性与关键规则

1. 存储基础信息

特性	详细说明
存储位置	用户浏览器的会话 Cookie 中（非持久化 Cookie，默认关闭浏览器后失效；可通过配置设置过期时间）
数据支持	仅支持 JSON 可序列化的基础类型（ <code>str</code> 、 <code>int</code> 、 <code>float</code> 、 <code>list</code> 、 <code>dict</code> 、 <code>bool</code> 、 <code>None</code> ），且数据需可被 Cookie 兼容（字符长度有限制）
生命周期	浏览器会话周期：默认关闭浏览器后 Cookie 失效，数据丢失；可手动设置过期时间，延长至会话外保留
共享范围	同一用户的同一浏览器下，所有标签页共享；不跨浏览器、不跨设备、不跨用户（不同浏览器 Cookie 独立）
依赖条件	必须在 <code>ui.run()</code> 中指定 <code>storage_secret</code> 参数（用于加密 Cookie 数据，防止篡改和伪造），否则无法使用

特性	详细说明
容量限制	受浏览器 Cookie 容量限制（单 Cookie 通常不超过 4KB，同一域名下总 Cookie 容量约 10-40KB），仅适合存储轻量数据

2. 关键使用规则

- `storage_secret` **必选**：依赖加密 Cookie 确保数据安全，必须在 `ui.run()` 中传入 `storage_secret`（复杂随机字符串），否则抛出错误；
- 数据限制严格**：不仅需 JSON 序列化，还受 Cookie 字符长度和格式限制，避免存储复杂字典或长字符串；
- 写入时机约束**：仅能在“响应发送前”写入数据（`write only before response`），页面加载完成后的异步操作中写入可能失效；
- 安全性有限**：Cookie 数据可被用户手动查看或修改（即使加密也存在破解风险），严禁存储敏感数据（如密码、令牌、个人隐私信息）；
- 与 `app.storage.user` 的关系**：`app.storage.browser['id']` 是 `app.storage.user` 的用户身份标识来源，二者配合实现“浏览器标识→用户存储”的关联。

三、核心配置与基础用法

1. 前置配置：指定 `storage_secret`

使用 `app.storage.browser` 前，必须在 `ui.run()` 中设置 `storage_secret`（用于加密 Cookie），示例：

```
from nicegui import app, ui

# 生产环境建议使用复杂随机字符串（如 secrets.token_hex(16)）
ui.run(storage_secret='your_secure_browser_secret_123')
```

2. 基础操作 API

`app.storage.browser` 的 API 与 Python 字典一致，支持键值对增删改查，核心操作如下：

操作	语法	功能描述	示例
读取值	<code>app.storage.browser[key]</code> 或 <code>app.storage.browser.get(key, default)</code>	从 Cookie 中读取值， <code>get</code> 支持默认值（键不存在时返回）	<code>mode = app.storage.browser.get('view_mode', 'list')</code>
写入 / 更新值	<code>app.storage.browser[key] = value</code>	写入值到 Cookie（自动序列化），覆盖已有键	<code>app.storage.browser['theme'] = 'light'</code>
删除值	<code>del app.storage.browser[key]</code> 或 <code>app.storage.browser.pop(key, default)</code>	从 Cookie 中删除指定键	<code>app.storage.browser.pop('temp_data', None)</code>

操作	语法	功能描述	示例
检查键是否存在	<code>key in app.storage.browser</code>	判断键是否存在于 Cookie 中	<code>if 'session_id' in app.storage.browser: ...</code>
清空存储	<code>app.storage.browser.clear()</code>	清空当前应用相关的所有 Cookie 数据 (谨慎使用)	<code>app.storage.browser.clear()</code>
获取长度	<code>len(app.storage.browser)</code>	返回 Cookie 中存储的键值对数量	<code>count = len(app.storage.browser)</code>

3. 扩展配置：设置 Cookie 过期时间

默认 Cookie 为会话级（关闭浏览器失效），可通过 `app.storage.browser.expires` 设置过期时间（单位：秒），实现长期保留：

```
from nicegui import app, ui

# 设置 Cookie 7 天后过期 (7*24*3600 秒)
app.storage.browser.expires = 604800

@ui.page('/')
def index():
    # 写入的数据会保留 7 天，即使关闭浏览器
    app.storage.browser['prefer_lang'] = 'zh-CN'
    ui.label(f'偏好语言: {app.storage.browser["prefer_lang"]}')

ui.run(storage_secret='expire_secret_456')
```

4. 基础使用示例

```
from nicegui import app, ui

# 配置 Cookie 过期时间为 1 天 (86400 秒)
app.storage.browser.expires = 86400

@ui.page('/')
def index():
    # 初始化浏览器存储的视图模式（默认列表模式）
    view_mode = app.storage.browser.get('view_mode', 'list')

    # 展示当前视图模式
    mode_label = ui.label(f'当前视图模式: {view_mode}')
```

```

# 切换视图模式（写入浏览器存储，跨标签页共享）
def switch_mode():
    new_mode = 'grid' if view_mode == 'list' else 'list'
    app.storage.browser['view_mode'] = new_mode
    mode_label.set_text(f'当前视图模式: {new_mode}')
    ui.notify(f'视图模式已切换为 {new_mode}，跨标签页同步')

# 清除视图模式设置
def clear_mode():
    if 'view_mode' in app.storage.browser:
        del app.storage.browser['view_mode']
        mode_label.set_text('当前视图模式: list（默认）')
        ui.notify('视图模式设置已清除')

ui.button('切换视图模式', on_click=switch_mode)
ui.button('清除设置', on_click=clear_mode)
ui.button('打开新标签页测试', on_click=lambda: ui.navigate.to('/', new_tab=True))

ui.run(storage_secret='browser_demo_secret_789')

```

效果：切换视图模式后，打开新标签页访问应用，视图模式与原标签页一致（跨标签页共享）；关闭浏览器 1 天内重新打开，设置仍保留；超过 1 天则恢复默认值。

四、典型应用场景与完整示例

场景 1：存储用户轻量偏好设置（跨标签页共享）

需求：用户设置页面布局、字体大小等轻量偏好，同一浏览器下所有标签页共享，短期保留（如 7 天），无需服务器存储。

```

from nicegui import app, ui

# 设置 Cookie 7 天过期
app.storage.browser.expires = 7 * 24 * 3600

@ui.page('/')
def index():
    # 从浏览器存储读取偏好设置（默认值兜底）
    preferences = app.storage.browser.get('preferences', {
        'layout': 'default',
        'font_size': 'medium'
    })

    # 展示当前偏好
    ui.label('=== 浏览器端偏好设置 ===').classes('text-xl font-bold')
    ui.label(f'页面布局: {preferences["layout"]}')
    ui.label(f'字体大小: {preferences["font_size"]}')

    # 切换布局
    def change_layout():
        new_layout = 'compact' if preferences['layout'] == 'default' else 'default'
        # 更新浏览器存储

```

```

app.storage.browser['preferences'] = {**preferences, 'layout': new_layout}
ui.notify(f'布局已切换为 {new_layout}, 跨标签页同步')
ui.navigate.reload() # 重载页面生效

# 切换字体大小
def change_font_size():
    sizes = ['small', 'medium', 'large']
    current_idx = sizes.index(preferences['font_size'])
    new_idx = (current_idx + 1) % 3
    new_size = sizes[new_idx]
    app.storage.browser['preferences'] = {**preferences, 'font_size': new_size}
    ui.notify(f'字体大小已切换为 {new_size}')
    ui.navigate.reload()

ui.button('切换布局', on_click=change_layout).classes('mt-4')
ui.button('切换字体大小', on_click=change_font_size).classes('mt-2')
ui.link('打开新标签页测试', '/', new_tab=True).classes('mt-2')

ui.run(storage_secret='preference_secret_123')

```

优势：偏好设置存储在客户端，不占用服务器资源；跨标签页共享，用户体验一致；7天过期平衡“保留时长”与“资源占用”。

场景 2：基于浏览器标识的唯一访问统计

需求：通过 `app.storage.browser['id']`（浏览器唯一标识）统计应用的独立访问用户数，无需关联用户账号。

```

from nicegui import app, ui
from collections import Counter
from datetime import datetime

# 全局计数器（存储在服务器内存，重启后重置；需持久化可改用 app.storage.general）
visitor_counter = Counter()
start_time = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

@ui.page('/')
def index():
    # 获取浏览器唯一标识（自动生成，存储在 Cookie 中）
    browser_id = app.storage.browser['id']

    # 新浏览器首次访问时，计数器自增
    if browser_id not in visitor_counter:
        visitor_counter[browser_id] = 1
    total_visits = sum(visitor_counter.values())
    unique_visitors = len(visitor_counter)

    # 展示统计信息
    ui.label('=== 应用访问统计 ===').classes('text-xl font-bold')
    ui.label(f'统计开始时间: {start_time}')
    ui.label(f'独立访问浏览器数: {unique_visitors}')
    ui.label(f'总访问次数: {total_visits}')
    ui.label(f'当前浏览器标识: {browser_id}').classes('text-sm text-gray-500')

```

```
ui.run(storage_secret='visitor_count_secret_456')
```

核心亮点：利用 `app.storage.browser['id']` 自动生成浏览器唯一标识，无需用户登录即可统计独立访问；数据存储在 Cookie 中，浏览器重启后标识不变（Cookie 未过期）。

场景 3：临时存储跨标签页交互状态

需求：用户在一个标签页设置临时筛选条件，切换到另一个标签页后仍可复用该条件，无需重新输入（轻量状态）。

```
from nicegui import app, ui

@ui.page('/')
def index():
    # 从浏览器存储读取筛选条件（默认空）
    filter_params = app.storage.browser.get('filter_params', {'keyword': '', 'category': 'all'})

    # 筛选条件输入组件
    keyword_input = ui.input('搜索关键词', value=filter_params['keyword']).classes('w-full')
    category_select = ui.select(['all', 'A', 'B', 'C'], value=filter_params['category'],
                                label='分类')

    # 保存筛选条件到浏览器存储
    def save_filter():
        new_params = {
            'keyword': keyword_input.value.strip(),
            'category': category_select.value
        }
        app.storage.browser['filter_params'] = new_params
        ui.notify('筛选条件已保存，跨标签页可用')

    # 应用筛选条件
    def apply_filter():
        ui.notify(f'应用筛选：关键词={filter_params["keyword"]}, 分类={filter_params["category"]}')

    # 清除筛选条件
    def clear_filter():
        app.storage.browser.pop('filter_params', None)
        keyword_input.value = ''
        category_select.value = 'all'
        ui.notify('筛选条件已清除')

    ui.button('保存筛选条件', on_click=save_filter).classes('mt-4')
    ui.button('应用筛选', on_click=apply_filter).classes('mt-2')
    ui.button('清除筛选', on_click=clear_filter).classes('mt-2')
    ui.link('新标签页打开（复用筛选条件）', '/', new_tab=True).classes('mt-2')

ui.run(storage_secret='filter_secret_789')
```

效果：保存筛选条件后，新标签页打开应用会自动加载该条件；关闭标签页后重新打开，条件仍保留（Cookie 未过期）；适合轻量、非关键的临时状态存储。

五、与其他存储类型的核心区别

对比维度	.browser	.user	.general	.client	.tab
存储位置	浏览器 Cookie	服务器（文件 / Redis）	服务器（文件 / Redis）	服务器内存	服务器内存
共享范围	同一用户 + 同一浏览器 + 所有标签页	同一用户（跨标签页 / 重启）	所有用户（全局共享）	同一客户端（标签页）	同一标签页
跨服务器重启	是（Cookie 未过期）	是（持久化）	是（持久化）	否	是（标签页未关）
数据类型支持	序列化类型（JSON+Cookie 兼容）	序列化类型（JSON 兼容）	序列化类型（JSON 兼容）	任意 Python 对象	任意 Python 对象
需 storage_secret	是	是	否	否	否
容量限制	极严格（4KB / 单 Cookie）	中等（文件 / Redis 限制）	中等（文件 / Redis 限制）	服务器内存限制	服务器内存限制
安全性	低（可被用户篡改）	中（服务器存储 + 加密）	中（服务器存储）	中（服务器内存）	中（服务器内存）
核心生命周期	浏览器会话（可延长）	用户长期周期	应用生命周期	页面访问周期	标签页会话周期

关键选型建议

- 需浏览器端本地存储、跨标签页共享、轻量非敏感数据 → 选 `app.storage.browser`；
- 需用户级持久化、服务器端安全存储、跨重启保留 → 选 `app.storage.user`（优先于 `browser`）；
- 需全局所有用户共享、应用级配置 → 选 `app.storage.general`；
- 需客户端临时存储、重载清空 → 选 `app.storage.client`；
- 需标签页独立存储、重载保留 → 选 `app.storage.tab`。

六、最佳实践与注意事项

1. 最佳实践

- **严格控制数据体量：**仅存储字符串、小字典等轻量数据（建议单键值不超过 1KB），避免触发 Cookie 容量限制；
- **避免敏感数据存储：**严禁存储密码、令牌、手机号、身份证号等敏感信息，Cookie 存在被篡改和泄露风险；
- **合理设置过期时间：**根据业务需求设置 `expires`，短期临时数据用会话级 Cookie（默认），长期轻量偏好可

设 7-30 天过期；

- `storage_secret` **安全配置**：生产环境使用 `secrets.token_hex(16)` 生成复杂随机字符串，通过环境变量传入（避免硬编码）：

```
import os
import secrets
from nicegui import ui

# 从环境变量读取，无则生成临时密钥（生产环境需固定）
storage_secret = os.getenv('NICEGUI_STORAGE_SECRET', secrets.token_hex(16))
ui.run(storage_secret=storage_secret)
```

- **优先使用** `app.storage.user`：若数据需要长期保留、安全性要求较高，即使是用户级数据，也优先选择 `app.storage.user`，而非 `browser`；
- **数据序列化优化**：存储字典时尽量精简字段，避免嵌套过深，减少序列化后的字符长度。

2. 注意事项

- **写入时机约束**：仅能在页面加载阶段（响应发送前）写入数据，异步回调（如定时器、按钮点击后的异步操作）中写入可能失败；
- **Cookie 兼容性问题**：部分浏览器对 Cookie 字符有特殊限制（如禁止特殊字符），存储前需确保数据无非法字符；
- **用户可手动清除**：用户可通过浏览器设置清除 Cookie，导致存储数据丢失，需做好默认值兜底；
- **加密并非绝对安全**：`storage_secret` 加密仅能防止普通篡改，无法抵御专业攻击，仍需避免敏感数据存储；
- **多域名问题**：若应用部署在多个域名下，Cookie 不跨域名共享，`app.storage.browser` 数据无法同步；
- **性能影响**：过多或过大的 Cookie 会增加 HTTP 请求头体积，影响页面加载速度，需控制存储的键值对数量。

七、总结

`app.storage.browser` 是 NiceGUI 针对“浏览器端轻量会话存储”场景的补充方案，核心优势在于**客户端本地存储、跨标签页共享、无服务器资源占用、响应速度快**。其适用场景集中在：

1. 轻量用户偏好设置（如视图模式、字体大小）；
2. 跨标签页临时状态（如筛选条件、会话标识）；
3. 浏览器唯一标识（用于匿名统计、临时关联）；
4. 无需服务器持久化的短期会话数据。

与其他存储类型相比，`app.storage.browser` 的核心特点是“浏览器端存储”，填补了“客户端本地、跨标签页共享”的空白，但受限于 Cookie 的容量、安全性和写入约束，使用场景相对狭窄。使用关键在于：明确其“轻量、非敏感、短期”的定位，严格控制数据体量和类型，避免滥用导致兼容性或安全性问题。